

Assignment 6: Grid Acceleration & Solid Textures

作者：王洵

1 实验原理

1.1 均匀网格加速求交

均匀体素网格 (uniform voxel grid) 将场景的轴对齐包围盒 (axis-aligned bounding box, AABB) 划分为 $n_x \times n_y \times n_z$ 个单元后, 射线与场景的求交分解为两步: 先沿射线得到穿过的单元序列, 再在各单元内对已登记的图元求交。

射线参数化为 $\mathbf{p}(t) = \mathbf{o} + t\mathbf{d}$ 。射线与网格外包围盒用 slab 法求得进入参数 t_{enter} 与离开参数 t_{exit} 。三维数字微分分析器 (3D DDA) 根据方向分量符号, 由各轴下一边界面参数 $t_{\text{next},x}, t_{\text{next},y}, t_{\text{next},z}$ 取最小值前进一步, 使遍历代价与穿过的体素数成正比, 而与场景图元总数无关。

对当前体素内每个有限图元求得交点参数 t 后, 接受条件为

$$t \in [t_{\text{cell}}^{\min}, t_{\text{cell}}^{\max}],$$

其中 $t_{\text{cell}}^{\min}$ 、 $t_{\text{cell}}^{\max}$ 为射线进入、离开该体素时的参数。跨越多格的图元可能在更远体素中产生更近的几何交点; 缺少区间约束时, 会把位于当前体素之外的交点当作本格命中。当已找到满足区间约束的最近交点且其 t 不超过当前体素出口时, 更远体素中的交点不可能更近, 故终止后续步进。阴影射线采用同一套空间索引与步进规则, 从而在判定光源可见性时减少对全体图元的遍历。

1.2 无限图元、变换展平与重复求交

无限平面等无有限包围盒的图元无法保守地放入有限网格单元。对这类图元在网格遍历之外单独求交, 再与网格内候选最近交点比较, 取二者中更近者作为最终命中, 使平面参与加速路径下的可见性判定。

含仿射变换的层次场景在栅格化时, 沿场景图向下累积变换矩阵 M , 将叶节点图元 (或其变换包装) 置于世界空间后插入重叠体素。对三角形, 先将三个顶点变换到世界坐标再取 AABB, 得到比变换局部盒八角点更紧的插入范围, 从而减少虚假占用。

同一图元落入多个体素时, 沿射线可能在不同单元中被重复测试。为每条射线施加一次性标记, 使同一图元在该次投射中只计算一次交点并缓存结果; 后续单元复用该 t 做区间判定, 从而降低重复求交次数, 且不改变可见性结果。

1.3 程序化实体纹理

程序化实体纹理 (procedural solid texture) 将材质属性定义为世界空间位置 $\mathbf{p}$ 的函数, 使颜色、高光与反射 / 透射系数随空间连续或分段变化。映射矩阵将 $\mathbf{p}$ 变到纹理坐标 $\mathbf{q} = M\mathbf{p}$ 。

三维棋盘格由各轴取整后的奇偶性判定: 令 $i = \lfloor q_x \rfloor$ 、 $j = \lfloor q_y \rfloor$ 、 $k = \lfloor q_z \rfloor$, 当 i, j, k 中奇数个为奇数时选取第二套子材质, 否则选取第一套, 从而在单位立方体上形成轴对齐的二值分区; 缩放与旋转由 M 控制格子大小与朝向。

Perlin 噪声给出伪随机、空间连续的标量场 $\text{noise}(\mathbf{q})$ 。分形叠加

$$N(\mathbf{q}) = \sum_{i=0}^{o-1} 2^{-i} \text{noise}(2^i \mathbf{q})$$

在低频起伏上叠加高频细节, 其中 o 为 octave 数。将 N 钳制并线性映射到 $[0, 1]$ 后, 在两套子材质的漫反射、镜面、反射、透射及折射率之间插值, 并使局部着色结果按同一权重混合, 从而环境光与二次射线所依赖的材质系数与着色点位置相符。

大理石纹路由正弦脉动叠加噪声扰动得到

$$M(\mathbf{q}) = \sin(f q_x + a N(\mathbf{q})),$$

其中频率 f 控制条带疏密，振幅 a 控制扭曲强度。木纹在圆柱半径 $r = \sqrt{q_y^2 + q_z^2}$ 上采用 $\sin(fr + aN(\mathbf{q}))$ ，形成沿轴向的年轮状环纹。上述标量经归一化后同样用于两子材质之间的插值，并支持棋盘格内再嵌套木纹等递归着色结构。

2 程序设计与实现

2.1 总体架构

系统在既有光线追踪器与均匀体素网格之上，将网格从占用可视化扩展为求交加速结构，并以程序化实体纹理扩展材质系统。主程序增加统计开关；统计模块在渲染前后分别初始化与打印计数。场景求交按是否存在网格、是否处于网格可视化模式，在暴力遍历、网格加速求交与体素占用着色之间切换；阴影射线走对应的加速或非加速路径。材质侧新增棋盘格、噪声、大理石与木纹；材质属性查询增加世界空间位置参数，使反射、透射与折射率随交点变化。

```
if (visualizeGrid && grid != NULL)
    return grid->intersect(ray, hit, tmin);
if (grid != NULL)
    return rayCastFast(ray, hit, tmin);
return rayCast(ray, hit, tmin);
```

2.2 光线追踪统计

渲染前按图像尺寸、场景包围盒与网格分辨率初始化计数器；结束后打印总时间、每像素平均射线数、每射线平均求交次数、每射线平均遍历体素数等。

```
RayTracingStats::Initialize(width, height, bbox, grid_nx, grid_ny, grid_nz);
// ... 渲染 ...
RayTracingStats::PrintStatistics();
```

计数规则：

- 主射线与反射 / 折射二次射线计入非阴影射线数；
- 每次发射阴影射线计入阴影射线数；
- 球体、三角形、平面的图元求交计入求交次数；
- 网格步进每进入下一单元计入遍历体素数。

2.3 网格加速求交

暴力求交对场景根节点遍历全部物体。网格加速求交先对无限图元列表求交，再初始化 3D DDA，并在各体素内对已登记图元求交；交点须落在当前体素参数区间内，且优于当前最优命中。若最优交点已不超过本体素出口，则提前结束步进。

```
beginIntersectionMarking();
for (/* 无限图元 */)
    infinite.getObject(i)->intersect(ray, bestHit, tmin);

grid->initializeRayMarch(mi, ray, tmin);
while (mi.getTMin() <= mi.getTExit()) {
    float cellTMin = mi.getTMin();
    float cellTMax = cellExitT(mi);
    // 对当前体素内图元：标记后求交或复用缓存 t
    if (hitInCell(t, cellTMin, cellTMax) && t < bestHit.getT())
        bestHit = candidate;
    if (found && bestHit.getT() <= cellTMax + CELL_T_EPSILON)
        return true;
    mi.nextCell();
}
```

阴影加速采用相同步进：遮挡判定遇交即返回；半透明阴影路径查询材质，取光路上最近透明交点。跨体素图元用一次性标记：首次求交缓存真实交点，后续体素只复用缓存的 t 做区间比较。

2.4 无限图元与变换展平

无限平面栅格化时写入无限图元列表，不占用体素：

```
void Plane::insertIntoGrid(Grid *g, Matrix *m) {
    if (g != NULL)
        g->addInfiniteObject(this);
}
```

变换节点将父链与本地矩阵合成为 $M_{\text{combined}} = M_{\text{parent}} \cdot M_{\text{local}}$ 后下传；有累积矩阵时为图元构造变换包装，在体素内将射线逆变换到局部空间求交，再将法线按 $(M^{-1})^T$ 变回世界空间。

```
Matrix combined = matrix;
if (m != NULL)
    combined = (*m) * matrix;
object->insertIntoGrid(g, &combined);
```

三角形在存在变换时先变换三顶点，再取世界空间 AABB 插入：

```
m->Transform(v0); m->Transform(v1); m->Transform(v2);
// 由变换后三顶点求 wmin、wmax
g->insertObjectInWorldAABB(wmin, wmax, this, m);
```

2.5 程序化实体纹理

棋盘格将交点经映射矩阵变到纹理空间，用取整与奇偶异或在两套子材质间切换，再由选中子材质完成局部着色；OpenGL 预览设置第一套子材质。

```
Vec3f p = mapToTextureSpace(mapping, worldPoint);
int ix = (int)floor(p.x()), iy = (int)floor(p.y()), iz = (int)floor(p.z());
if (procOdd(ix) ^ procOdd(iy) ^ procOdd(iz))
    return mat2;
return mat1;
```

噪声材质用分形叠加得到 N ，映射到 $[0, 1]$ 后对子材质属性与着色结果做线性插值：

```
for (int i = 0; i < octaves; i++) {
    sum += weight * PerlinNoise::noise(x, y, z);
    x *= 2.0; y *= 2.0; z *= 2.0;
    weight *= 0.5;
}
```

大理石按 $\sin(fx + aN)$ 计算混合权重；木纹在圆柱半径上按 $\sin(fr + aN)$ 计算。局部着色与反射、折射、透明阴影路径均按交点位置读取空间变化的 $\mathbf{k}_d$ 、 $\mathbf{k}_r$ 、 $\mathbf{k}_t$ 与折射率。

```
float v = sinf(freq * p.x() + amplitude * (float)n);
return clamp01(v * 0.5f + 0.5f);
```

3 实验结果

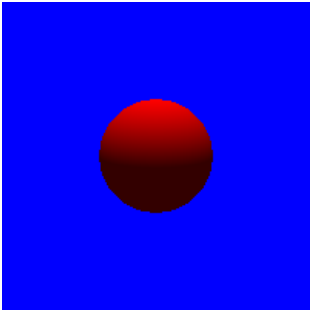
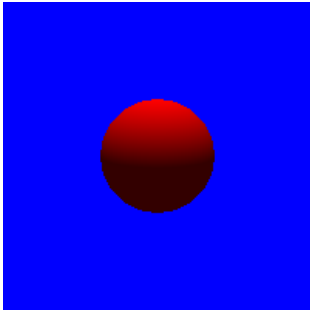
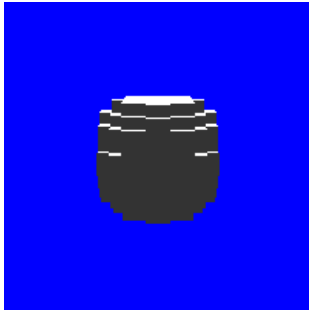
测试命令形如：

```
# 单球场景；-stats 打印统计，-size 图像分辨率
raytracer -input <单球场景> -output output6_01a.tga -size 200 200 -stats
# 启用 10×10×10 网格；-grid 体素分辨率
raytracer -input <单球场景> -output output6_01b.tga -size 200 200 -grid 10 10 10 -stats
# 棋盘格场景；-shadows 投射阴影
```

```
raytracer -input <棋盘格场景> -size 200 200 -output output6_13.tga -shadows
# 大理石立方体场景
raytracer -input <大理石立方体场景> -size 300 300 -output output6_15.tga
```

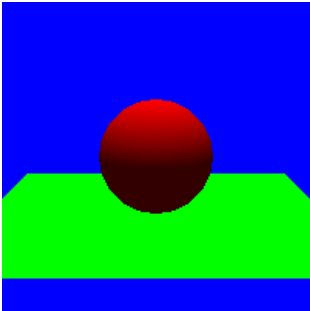
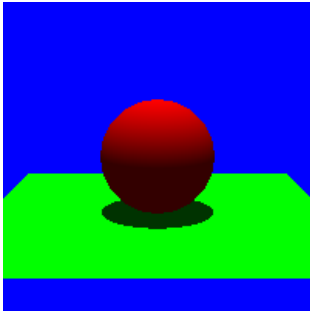
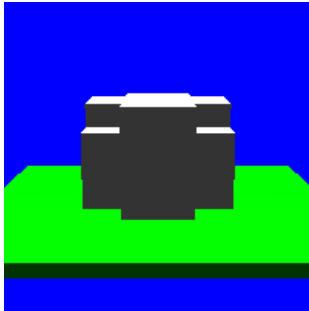
无网格时，单球场景在 200×200 下非阴影射线数为 40000，每射线平均求交约 1.0；含三个图元的场景约 3.0，开启阴影后阴影射线与求交次数随之上升。启用 10^3 网格后，单球每射线平均求交约降至 0.1（含标记去重）；约二百面三角网格的兔子在 $10 \times 10 \times 7$ 下由约 201 降至约 6.1。简单场景网格可能因步进开销略增总时间，复杂三角网格则显著加速；阴影与递归射线会放大加速收益。有无网格下着色图像相同。

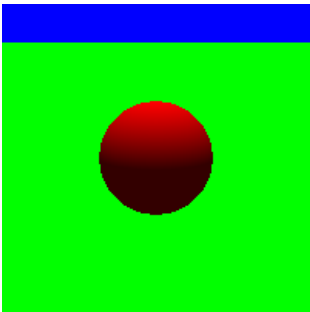
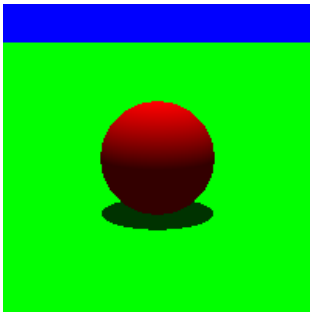
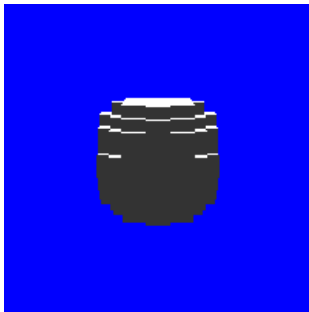
3.1 单球：有无网格与体素可视化

无网格着色	$10 \times 10 \times 10$ 网格着色	同分辨率体素占用可视化
		
暴力遍历求交	加速路径，明暗与左图一致	球体对应体素包络

有无网格下球体明暗与轮廓相同；可视化显示球体对应的体素包络，说明加速路径未改变可见性结果。

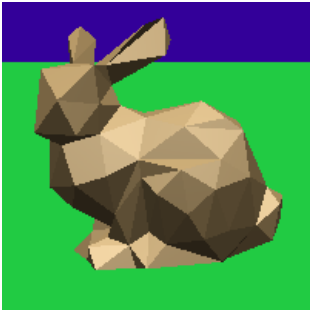
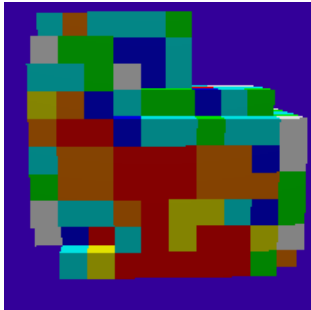
3.2 多图元、平面与阴影

球体 + 三角形 无阴影	有网格 + 阴影	体素可视化
		
局部着色基准	阴影落点清晰	占用贴合有限图元

球体 + 平面 有网格无阴影	有网格 + 阴影	体素可视化
		
平面参与可见性	平面正确遮挡	平面不进入体素列表

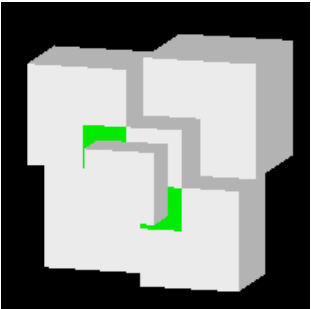
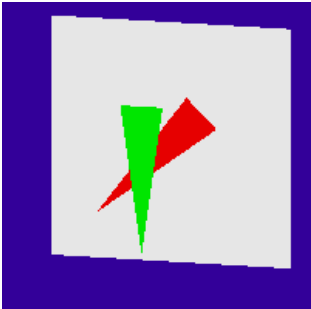
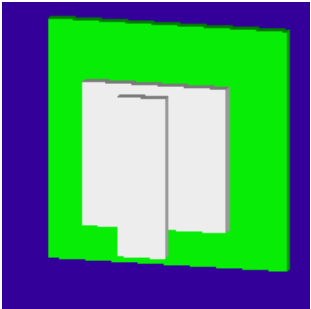
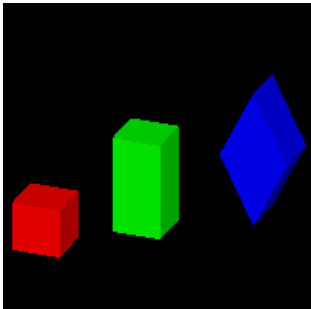
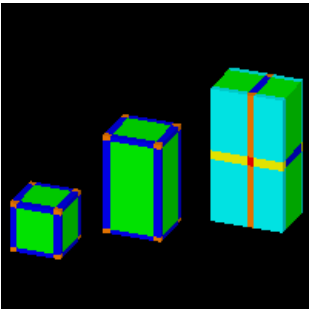
阴影落点清晰；无限平面单独求交后仍正确参与遮挡。体素图中占用区域贴合有限图元，平面本身不进入体素列表。

3.3 Stanford bunny：面数与网格加速

约 200 面 网格着色	有阴影	体素可视化
		
约 1k 面 网格 + 阴影	约 5k 面	约 40k 面
		

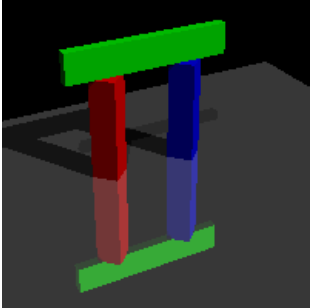
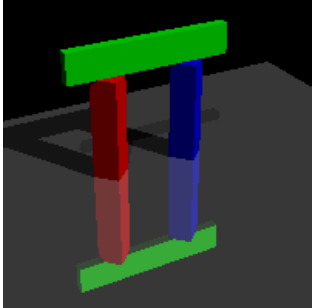
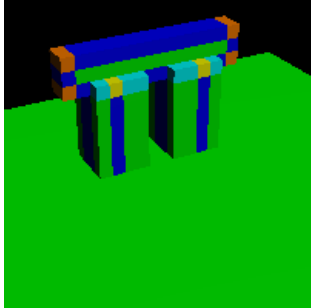
随三角面数增加，网格加速使求交次数与渲染时间明显下降；体素包络贴合兔子外形。面数越高，网格分辨率通常也相应提高，以控制单体内图元密度。

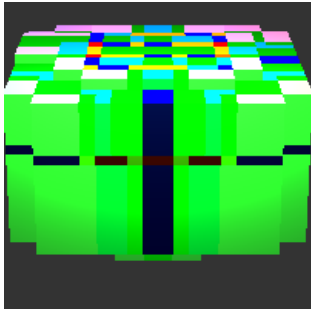
3.4 变换层级栅格化

缩放平移 网格着色	体素可视化	旋转三角形 网格着色	体素可视化
			
嵌套变换 网格着色		体素可视化	
			
有无网格着色一致		紧 AABB 占用更贴合轮廓	

矩阵累积展平后，有无网格下着色相同；三角形对变换后三顶点取紧 AABB，体素占用更贴合物体轮廓。

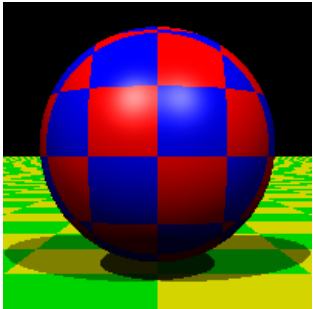
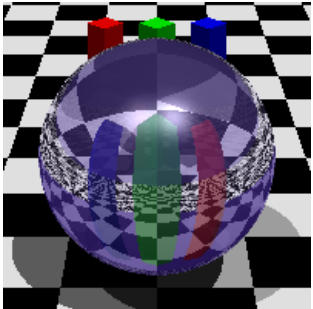
3.5 反射、折射与网格

镜面地板 无网格	有网格	体素可视化
		
阴影 + 一次反弹	图像与左图相同	占用可视化

多面宝石 无网格	有网格	体素可视化
		
阴影 + 多次反弹 + 背面色	图像与左图相同	占用可视化

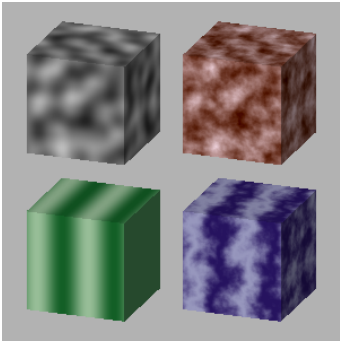
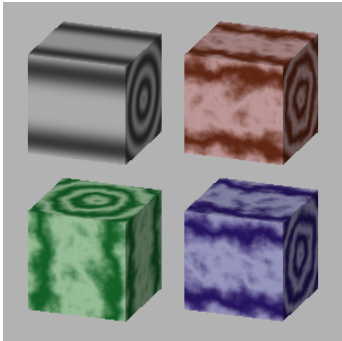
间接光照下有无网格图像相同，说明加速同时作用于主射线与阴影、反射 / 折射二次射线；统计量反映复杂路径下加速收益更明显。

3.6 棋盘格与玻璃球

棋盘格地面 & 球体（阴影）	玻璃球 + 棋盘环境（阴影、多次反弹、网格）
	
轴对齐实体棋盘	球内映出棋盘与环境

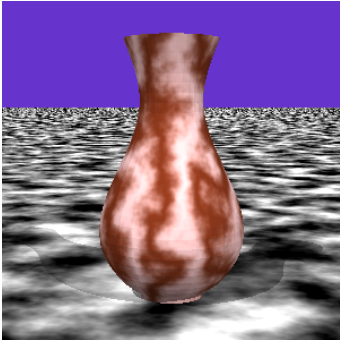
地面呈现轴对齐实体棋盘；玻璃球在递归反射 / 折射下映出棋盘与环境，空间变化的反射、透射系数在二次射线上同样生效。

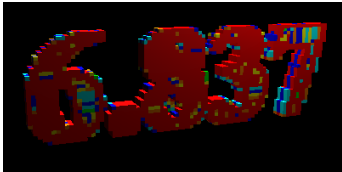
3.7 大理石与木纹立方体

噪声 / 大理石立方体 300 × 300	木纹立方体 300 × 300
	
噪声混合与大理石条带	圆柱年轮状木纹

表面可见噪声混合与大理石条带、木纹年轮；频率与振幅分别控制纹路疏密与扭曲强弱。

3.8 花瓶与标志网格

大理石花瓶 & 噪声底座 网格着色	体素可视化
	
纹路连续，阴影正确	占用贴合花瓶外形

复杂 procedural & 阴影 / 反弹	体素可视化
	
棋盘 / 木纹嵌套与反射折射	细长几何的占用分布

花瓶与底座在网格加速与阴影下纹路连续；标志场景将棋盘、木纹等嵌套材质与反射折射结合，体素图反映细长几何的占用分布。

4 问题记录

最初实现的网格加速求交在步进过程中直接采纳体素内图元的交点，但没有检查交点参数是否落在当前单元区间内。而跨多格的球体或三角形常在更远体素中算出更近的几何 t ，所以被误当作本格命中，导致有无网格情况下图像不一致。检查实现要点后，加入 $t \in [t_{\text{cell}}^{\min}, t_{\text{cell}}^{\max}]$ 判定，并在最近交点已不超过本格出口时提前终止步进，解决了问题。后期发现单元边界处浮点误差仍会导致偶发漏检或过早结束，表现为开启网格后地板缺失或出现斑点。解决方法是将区间比较与 early exit 放宽为带 ϵ 的容差，并对薄片图元的世界 AABB 略作膨胀后再插入，减少边界漏插。

后期实现标记去重时，调试信息表明在第一次访问某图元后便跳过了后续体素中的同指针对象。原因是球体占用大量相邻体素时，会先在较远单元中求到落在单元外的交点并打上标记，而应命中的近处单元反而被跳过，故而网格路径出现缺块。修改为求交后缓存真实 Hit，后续单元则复用缓存的 t 做区间判定；另外若第一次求交以当前 bestHit 构造候选上界，更近的真实交点会被 intersect 内部拒绝而无法写入缓存，所以缓存求交改用无穷远 / tmax 上界，问题解决。

平面阴影路径一度始终判为不透明遮挡。原因是平面阴影求交命中后未写回材质，透明阴影衰减无法读取平面材质。补上写回后正常。

变换球体场景中，有网格着色出现异常黑带，体素密度图也缺少应有的重叠着色。原因是变换后球体仍按局部距离公式插入，未按世界空间 AABB 保守登记重叠体素，导致加速路径漏测或栅格化偏稀。改为与通用有限图元一样进行包围盒插入后，着色与密度可视化恢复正常。此外，发现旋转三角形如果只变换局部 AABB 的八角点，世界盒会较大、空体素偏多；后期修正为先变换三顶点再取紧 AABB 插入，使占用轮廓更贴合几何模型。

棋盘格、大理石与木纹场景一度整体发糊，几乎看不出空间变化。对照后发现局部着色构造 shadedHit 时忘记调用 set(..., ray)，intersectionPoint 默认为原点附近，程序化材质的 Shade 在错误点采样。补上交点写入后，棋盘边界与噪声 / 条纹变清晰，问题解决。

标志场景渲染时画面水平方向曾被裁切拉伸。原因是 generateCameraRay 在宽高比大于 1 时压缩了错误的参数，调整后解决。

标志场景地板本应是清晰棋盘，却发糊且调节棋盘的 Uniform 参数几乎无效。原因是棋盘格的两套子材质均为带 logo 专用旋转与大缩放的木质材质，地板命中棋盘后跑的是木纹自身的映射矩阵采样，棋盘尺度没有起作用。修改方案：选中 Wood 子材质时，用棋盘格自己的纹理坐标计算木纹权重并提高频率，再在两套底色间插值，从而保留木纹条纹同时让棋盘格控制分区尺度。

编译老问题：课程解析器中 new (Light*)[n] / new (Material*)[n] 在当前 MinGW 的 g++ 下无法编译，改为 new Light*[n] / new Material*[n] 后解决。

5 总结

在既有光线追踪与均匀体素网格的基础上，通过 3D DDA 步进对当前单元内图元求交，并结合单元参数区间约束与提前终止，将网格用于主射线与阴影射线的空间加速；无限平面单独求交后与网格候选比较，变换层级经矩阵累积展平后插入体素，使复杂三角网格场景的求交次数明显下降。同时以棋盘格、分形 Perlin 噪声、大理石与木纹等程序化实体纹理描述空间变化的材质属性，并在局部着色与二次射线路径上交点坐标插值或切换子材质，使渲染结果在有无加速、有无阴影及不同网格分辨率下与几何与材质预期相符。

附录 (TODO list 记录)

- ☒ 在主程序中接入光线追踪统计：按图像尺寸与网格分辨率初始化，统计非阴影 / 阴影射线、图元求交与体素遍历，结束后打印汇总量
- ☒ 实现暴力求交与网格加速求交两套路径；加速路径对当前体素内图元求交，做单元区间判定与提前终止，阴影射线同样加速
- ☒ 用一次性标记缓存跨体素图元的交点，避免同一射线内重复求交
- ☒ 将无限平面写入网格的无限图元列表，加速求交时与网格内候选比较取更近者
- ☒ 变换节点级联累积矩阵后下传；有变换时包装图元做局部求交；三角形对变换后三顶点取更紧世界空间包围盒再插入
- ☒ 对各测试场景对比有无网格、有无阴影下的统计量，并文字总结性能规律
- ☒ 实现三维棋盘格材质：映射到纹理空间后按取整与奇偶选材，着色交给选中子材质
- ☒ 集成 Perlin 噪声并实现噪声材质：分形叠加后在两子材质间插值；材质属性查询带世界空间位置
- ☒ 实现大理石材质：按正弦加噪声公式混合两子材质
- ☒ 实现木纹材质：按圆柱半径上的正弦加噪声混合两子材质，并支持棋盘格内嵌木纹

Extra Credit

- ☐ 实验其他加速数据结构（嵌套网格、八叉树、包围体层次等）
- ☐ 实现分布式光线追踪效果
- ☐ 创建含复杂几何、有趣光照或程序化实体纹理的新场景
- ☐ 二维纹理映射
- ☐ Phong 法线插值、凹凸贴图